

KANBIEN PLATFORM ARCHITECTURE

TypeScript Architecture Guide

A first-principles explanation of TypeScript and an enterprise-grade monorepo setup for Kanbien.

Generated June 2026

Table of Contents

My architectural recommendation for Kanbien	4
1. What TypeScript is	5
2. Why TypeScript exists	6
3. How TypeScript differs from JavaScript	7
4. What problems TypeScript solves	8
5. What problems TypeScript does not solve	9
6. What tradeoffs TypeScript introduces	10
7. Why enterprise software teams commonly choose TypeScript	11
8. When TypeScript is the wrong choice	12
Kanbien monorepo structure	13
apps/	13
packages/	15
platform/	17
harness/	19
tools/	20
infra/	22
TypeScript configuration setup	24
Recommended config files	24
Root tsconfig.base.json	24
Root tsconfig.json	25
Package-level tsconfig.json	25
App-level tsconfig.json	26
Path aliases	27
Project references	27
Strict mode	28
Module resolution	28
ESM vs CommonJS	29
Linting	29
Formatting	30
Type checking	30
Build outputs	31
Recommended clean setup for Kanbien	32
Package manager	32
Monorepo tooling	32
TypeScript compiler settings	32
Linting	33
Formatting	33
Testing	33

Code generation	33
CI checks	34
How TypeScript should support the entity-builder idea.....	35
What TypeScript enforces	35
What TypeScript cannot enforce alone	36
Worked User example	37
1. Entity definition.....	37
2. Inferred TypeScript type	38
3. Generated validation schema	39
4. API input/output contract.....	40
5. Persistence model	41
6. Frontend form config	42
7. Frontend table config	42
8. Generated tests	43
The key design lesson	45

My architectural recommendation for Kanbien

For a serious greenfield SaaS platform, TypeScript should not be treated as "JavaScript with nicer autocomplete." It should be treated as a **contract system** across product modules, APIs, entity metadata, validation, generated artifacts, and deployment tooling.

For Kanbien, I would set up TypeScript around five principles:

1. **Strict by default.** New platform code should not start permissive and "tighten later."
2. **Packages are boundaries.** A folder is not a boundary; a package with its own `package.json`, `tsconfig`, lint rules, tests, and public exports is a boundary.
3. **Apps compose; packages contain reusable logic.** Apps should be thin composition roots.
4. **The entity-builder should be a platform primitive.** Entity definitions should drive types, validation, API contracts, database metadata, UI metadata, and tests.
5. **Generated code must be checked, not trusted.** Code generation is useful only if CI proves the generated output still type-checks, validates at runtime, and matches expected contracts.

TypeScript's official documentation describes it as building on JavaScript by adding type syntax for type-checking and rich editor tooling, and the current stable TypeScript line as of the latest official release notes is TypeScript 6.0, with TypeScript 7 native compiler previews in progress. For Kanbien, I would use the stable TypeScript 6.x line first and evaluate the 7.x native compiler separately once stable. (devblogs.microsoft.com)

1. What TypeScript is

TypeScript is a programming language and toolchain that sits on top of JavaScript.

More precisely:

JavaScript is the runtime language. Browsers, Node.js, serverless runtimes, edge runtimes, and many build tools run JavaScript.

TypeScript is JavaScript plus a static type system. You write `.ts` or `.tsx` files. The TypeScript compiler checks whether your code appears type-correct before it runs, then emits JavaScript or lets another build tool emit JavaScript.

A simple example:

```
function sendEmail(to: string, subject: string): void {
  console.log(`Sending ${subject} to ${to}`);
}

sendEmail("alice@example.com", "Welcome");
sendEmail(123, "Welcome"); // TypeScript error
```

The important thing: the type annotations are for tooling and checking. They do not become runtime checks by default. TypeScript's own docs say type annotations are erased and do not change runtime behavior. ([typescriptlang.org](https://www.typescriptlang.org))

So this:

```
function add(a: number, b: number): number {
  return a + b;
}
```

becomes roughly this JavaScript:

```
function add(a, b) {
  return a + b;
}
```

That distinction matters a lot for APIs. TypeScript can tell your code that `request.body.email` should be a string, but it cannot prove that an actual HTTP request from the outside world contains a valid email unless you validate it at runtime.

2. Why TypeScript exists

JavaScript is extremely flexible. That flexibility helped it become universal, but it creates problems at scale.

In a small script, this is fine:

```
const user = getUser();
console.log(user.email.toLowerCase());
```

In a large platform, you need to know:

```
type User = {
  id: string;
  email: string;
  displayName: string;
  createdAt: Date;
};
```

TypeScript exists because large JavaScript systems need:

- earlier feedback before code runs;
- safer refactoring;
- clearer API contracts;
- better editor navigation;
- reusable domain models;
- fewer accidental mismatches between modules;
- better onboarding for teams;
- machine-checkable boundaries between parts of the system.

The TypeScript docs frame the core value similarly: a static type-checker helps find bugs before code runs, and the type information also powers editor tooling such as autocomplete, navigation, quick fixes, and refactoring. (typescriptlang.org)

3. How TypeScript differs from JavaScript

JavaScript is dynamically typed. Values have runtime behavior, but variables and function parameters do not have enforced static types.

```
function createUser(email) {  
  return { email };  
}  
  
createUser("a@example.com");  
createUser(123);  
createUser(null);
```

All three calls are allowed by JavaScript. Some may fail later, but JavaScript itself does not stop them at development time.

TypeScript lets you describe the expected shapes:

```
function createUser(email: string) {  
  return { email };  
}  
  
createUser("a@example.com");  
createUser(123); // error  
createUser(null); // error in strict mode
```

TypeScript also adds several type-system features that do not exist in JavaScript:

```
type UserId = string;  
  
interface User {  
  id: UserId;  
  email: string;  
}  
  
type CreateUserInput = {  
  email: string;  
  displayName: string;  
};  
  
type ApiResponse<T> =  
  | { ok: true; data: T }  
  | { ok: false; error: string };
```

But runtime execution is still JavaScript. TypeScript is not a new runtime, not a database schema, not an API gateway, and not a validation engine.

There is now also native TypeScript type-stripping support in modern Node.js, but Node's documentation is explicit that it strips erasable TypeScript syntax and performs **no type checking**. That is useful for some scripts, but it does not replace `tsc` or CI type-checking for Kanbien. (nodejs.org)

4. What problems TypeScript solves

TypeScript is especially strong at preventing **interface mismatch** problems.

Examples:

```
type User = {
  id: string;
  email: string;
};

function sendWelcomeEmail(user: User) {
  sendEmail(user.email);
}

sendWelcomeEmail({ id: "u_123" });
// error: email is missing
```

It helps with refactoring:

```
type User = {
  id: string;
  primaryEmail: string;
};
```

If you rename `email` to `primaryEmail`, TypeScript can point to every place that still expects `email`.

It helps encode legal states:

```
type Invoice =
  | { status: "draft"; id: string }
  | { status: "sent"; id: string; sentAt: Date }
  | { status: "paid"; id: string; paidAt: Date };

function canEdit(invoice: Invoice): boolean {
  return invoice.status === "draft";
}
```

It helps keep API contracts aligned:

```
type CreateUserRequest = {
  email: string;
  displayName: string;
};

type CreateUserResponse = {
  id: string;
  email: string;
  displayName: string;
};
```

It helps with generated code. If Kanbien generates a `UserCreateInput` type from a `User` entity definition, every API handler, form, test, and repository function can depend on the same generated contract.

The biggest platform-level value is not that individual functions are safer. It is that **changes propagate visibly**. If a core entity changes, TypeScript can make the blast radius obvious.

5. What problems TypeScript does not solve

TypeScript does **not** solve runtime validation.

This is unsafe:

```
type CreateUserInput = {
  email: string;
  displayName: string;
};

app.post("/users", async (req, res) => {
  const input = req.body as CreateUserInput;
  await createUser(input);
});
```

The `as CreateUserInput` assertion does not validate anything. It just tells the compiler, "trust me."

For untrusted input, you need runtime validation:

```
const input = UserCreateInputSchema.parse(req.body);
await createUser(input);
```

A library such as Zod exists for exactly this reason: it provides runtime schemas and can infer TypeScript types from those schemas. Zod describes itself as a TypeScript-first validation library for defining schemas and parsing untrusted data. (zod.dev)

TypeScript also does not solve:

- bad product requirements;
- incorrect business logic;
- race conditions;
- authorization mistakes;
- database migration mistakes;
- distributed-system failure modes;
- performance bottlenecks;
- secrets management;
- poor test design;
- poor observability;
- all security issues;
- incorrect type assertions;
- use of `any`;
- stale generated code.

A useful rule: **TypeScript proves consistency between code artifacts. It does not prove reality.**

Reality still needs validation, tests, monitoring, migrations, and operational discipline.

6. What tradeoffs TypeScript introduces

TypeScript adds real value, but it is not free.

The tradeoffs:

More tooling. You now care about `tsconfig.json`, compiler versions, module resolution, declarations, lint rules, editor behavior, and build output.

More build complexity. Some packages need emitted JavaScript. Some apps rely on bundlers. Some internal packages can be consumed as source. If you do not standardize this, a monorepo gets messy fast.

False confidence. A type assertion can lie:

```
const user = JSON.parse(raw) as User;
```

That compiles even if `raw` is garbage.

Type-system complexity. Advanced generics, conditional types, inferred types, branded types, and mapped types are powerful but can become unreadable.

Slower feedback if configured poorly. Type-aware linting and monorepo type-checking can be expensive. The `typescript-eslint` docs call out that typed linting has a performance cost because it asks TypeScript to build type information before linting. (typescript-eslint.io)

ESM/CommonJS friction. JavaScript has two major module systems in active use: ESM and CommonJS. TypeScript supports both, but the interop can be painful. TypeScript's module docs identify ESM and CommonJS as the two most important module systems today, and Node's docs define ESM as the official standard module format for JavaScript reuse. (typescriptlang.org)

Compiler upgrades require governance. TypeScript evolves. A serious platform should pin versions, run automated upgrade PRs, and treat compiler upgrades like platform changes.

7. Why enterprise software teams commonly choose TypeScript

Enterprise teams choose TypeScript because it gives JavaScript a stronger engineering model without abandoning the JavaScript ecosystem.

The common reasons:

One language across frontend and backend. A team can share types, validation schemas, SDKs, and UI models between web apps, APIs, workers, and tools.

Better API discipline. Request and response models can be checked across layers.

Better refactoring. Large-scale renames and contract changes become safer.

Better onboarding. Types document intent directly in code.

Better editor experience. Autocomplete, go-to-definition, find references, quick fixes, and inline errors improve daily productivity.

Better monorepo ergonomics. Shared packages can expose typed public APIs. Project references can divide large TypeScript programs into smaller pieces, improving build times and enforcing logical separation. (typescriptlang.org)

Better compatibility with modern web tooling. React, Next.js, Vite, Node, serverless tooling, API clients, design systems, and test frameworks all have strong TypeScript support.

For Kanbien, the biggest reason is this: your entity-builder vision needs a language that can connect metadata, generated types, validation, API contracts, forms, tables, tests, and repositories. TypeScript is a very good fit for that.

8. When TypeScript is the wrong choice

TypeScript is the wrong choice when the team will not use it seriously.

Bad TypeScript is worse than honest JavaScript:

```
const data: any = await getData();  
const user = data as any as User;
```

That creates bureaucracy without safety.

TypeScript may be a poor fit when:

- the codebase is tiny and disposable;
- the team is not willing to learn the type system;
- the platform is mostly in another strongly typed ecosystem already;
- runtime validation is mistaken for compile-time typing;
- developers will bypass the type system with `any` and assertions;
- the product needs extreme runtime dynamism where static modeling adds little;
- the build/deployment environment cannot tolerate the extra tooling;
- the organization will not govern compiler settings, generated code, and package boundaries.

For Kanbien, assuming a serious SaaS platform with entity modeling, APIs, frontend configuration, governance, and generated artifacts, TypeScript is a strong choice.

Kanbien monorepo structure

I would structure Kanbien like this:

```
kanbien/  
  apps/  
  packages/  
  platform/  
  harness/  
  tools/  
  infra/
```

Think of these as architectural zones, not just folders.

A good dependency direction is:

```
apps  
└─ depend on packages and platform  
  
packages  
└─ depend on platform, never on apps  
  
platform  
└─ depends on very little; foundational primitives  
  
harness  
└─ test-only support; production code must not depend on it  
  
tools  
└─ repo automation, generators, CLIs; generated outputs may be used by production code  
  
infra  
└─ deployment/IaC; should consume config and generated metadata, not app internals
```

In other words:

```
apps ────> packages ────> platform  
└─┬─┘  
   └─ test-only ► harness  
  
tools ────> platform / package metadata  
infra ────> config / generated deployment metadata
```

Nx is a strong fit here because it supports package-manager workspaces, project discovery, dependency graphs, affected builds, and boundary enforcement. Nx's docs describe using package-manager workspaces for projects, enforcing boundaries through rules, and running only affected tasks in CI. (nx.dev)

apps/

Purpose

Deployable applications.

Examples:

```
apps/  
  web/  
  api/  
  worker/  
  admin/  
  public-site/
```

Apps are composition roots. They assemble platform services, product packages, routes, UI, environment config, logging, telemetry, database clients, and deployment-specific behavior.

TypeScript code that should live there

- HTTP server bootstrap.
- Web app entry points.
- Route registration.
- App-specific page composition.
- Worker bootstrap.
- Environment wiring.
- Dependency injection.
- Framework-specific adapters.
- App-level configuration.
- App-level smoke tests.

Example:

```
import { createUserRoutes } from "@kanbien/users-api";
import { createLogger } from "@kanbien/platform-observability";
import { createDb } from "@kanbien/platform-db";

const logger = createLogger();
const db = createDb();

export const app = createHttpApp({
  routes: [createUserRoutes({ db, logger })],
});
```

What should not live there

- Reusable business rules.
- Entity definitions.
- Shared API contracts.
- Shared UI components.
- Database schema definitions.
- Cross-app utility functions.
- Domain logic that another app might need.
- Test factories used by multiple packages.

If logic starts in `apps/api/src/users/create-user.ts` and another app soon needs it, it probably belongs in `packages/users` or `platform/entity`.

Dependency rules

Apps may depend on:

- `packages/*`
- `platform/*`
- generated packages
- app-local code
- `harness/*` only from tests

Apps must not be depended on by:

- packages
- platform
- tools
- other apps, except through published API clients or contracts

Testing approach

- Unit tests for app-specific wiring.
- Integration tests for routes.
- Contract tests against generated API schemas.
- E2E tests through `harness/e2e`.
- Smoke tests for deployment health.

Most business logic tests should live below `apps`, closer to the package that owns the logic.

Build approach

Frontend apps usually use a framework bundler. Backend apps can either:

- build with the framework/tooling;
- emit Node-compatible JavaScript with `tsc`;
- bundle with a server bundler if deployment benefits from that.

For browser-bundled apps, prefer `moduleResolution: "Bundler"`. For Node-emitted apps, prefer `module: "NodeNext"` and `moduleResolution: "NodeNext"`.

packages/

Purpose

Reusable product and domain packages.

Examples:

```
packages/  
  users/  
  users-api/  
  organizations/  
  billing/  
  permissions/  
  notifications/  
  ui/  
  api-client/  
  config-typescript/  
  config-eslint/
```

A package should have a clear public API. Other code should import from the package entry point, not random internal files.

Good:

```
import { createUser } from "@kanbien/users";
```

Bad:

```
import { createUser } from "@kanbien/users/src/internal/create-user";
```

TypeScript code that should live there

- Domain services.
- Application services.
- Shared UI components.
- API client code.
- Feature-specific contracts.
- Package-local validation.
- Package-local test helpers.
- Package-specific adapters.

Example:

```
export async function createUser(input: CreateUserInput, deps: CreateUserDeps) {  
  const parsed = UserCreateInputSchema.parse(input);  
  return deps.userRepository.create(parsed);  
}
```

What should not live there

- App bootstrapping.
- Deployment logic.
- Global platform primitives.
- Entity-builder engine internals.
- Raw infrastructure provisioning.
- Cross-cutting concerns that every package reimplements.

If five packages need the same primitive, it probably belongs in `platform/`.

Dependency rules

Packages may depend on:

- `platform/*`
- other `packages/*` when allowed by domain boundaries
- generated contracts

Packages must not depend on:

- `apps/*`
- `infra/*`
- `tools/*` at runtime
- `harness/*` outside tests

For example:

```
packages/users-api → packages/users  
packages/users   → platform/entity-runtime  
packages/users   → platform/db
```

But avoid:


```
packages/users → apps/api
```

Testing approach

- Unit tests for pure logic.
- Integration tests for repositories/adapters.
- Contract tests for API package boundaries.
- Type tests for public APIs.
- Snapshot tests for generated metadata only where useful.

Build approach

Each package should have its own `package.json`, `tsconfig.json`, `src/`, and `dist/`.

For buildable packages:

```
packages/users/  
  src/  
  dist/  
  package.json  
  tsconfig.json  
  project.json
```

Build with `tsc -b` for declarations and JavaScript, or use a library bundler only when bundling is actually needed. For most internal packages, `tsc` plus package exports is enough.

platform/

Purpose

Reusable platform primitives and engines.

Examples:

```
platform/  
  entity-core/  
  entity-runtime/  
  entity-codegen/  
  db/  
  api/  
  auth/  
  observability/  
  config/  
  errors/  
  events/  
  security/
```

This is where Kanbien's internal platform lives. It should be boring, stable, heavily tested, and carefully governed.

TypeScript code that should live there

- Entity-builder core types.
- Entity definition DSL.
- Code generation engine.
- Runtime validation adapters.
- API route abstractions.
- Database abstractions.

- Error/result primitives.
- Logging/metrics/tracing abstractions.
- Auth/session primitives.
- Event bus abstractions.
- Security policy primitives.

Example:

```
export type Result<T, E> =
  | { ok: true; value: T }
  | { ok: false; error: E };
```

What should not live there

- Product-specific business logic.
- App-specific route wiring.
- UI screens.
- One-off utilities.
- Domain packages disguised as platform code.
- Anything that changes every sprint because a product workflow changed.

A common failure mode: teams put everything in "platform" because it sounds important. Resist that. Platform should be stable leverage.

Dependency rules

Platform packages should depend on:

- other lower-level platform packages
- small, well-governed external libraries

Platform packages should not depend on:

- apps
- product packages, except perhaps generated-neutral contract packages in very controlled cases
- harness except tests
- infra

Example layering inside platform:

```
platform/entity-core
└─ no dependency on Zod, DB, React, HTTP

platform/entity-runtime
└─ may depend on entity-core and Zod

platform/entity-codegen
└─ may depend on entity-core, runtime schema libraries, filesystem tools

platform/api
└─ may depend on entity-runtime, errors, observability
```

Testing approach

- Heavy unit test coverage.
- Property-based tests where useful.
- Type-level tests for inference behavior.

- Compatibility tests for generated output.
- Golden-file tests for code generation.
- Mutation-style thinking around validation and security-sensitive code.

Build approach

Platform packages should be buildable, versionable internally, and produce declaration files.

No platform package should require app code to type-check.

harness/

Purpose

Testing and development support.

Examples:

```
harness/  
  testkit/  
  fixtures/  
  factories/  
  e2e/  
  contract/  
  fake-services/  
  storybook/
```

The harness makes testing easier without leaking into production.

TypeScript code that should live there

- Test data factories.
- Fake repositories.
- Contract-test runners.
- E2E utilities.
- API test clients.
- Seed data.
- Mock service workers.
- Test environment setup.
- Browser automation helpers.
- Visual regression setup.

Example:

```
export function makeUser(overrides: Partial<User> = {}): User {  
  return {  
    id: "user_test_123",  
    email: "test@example.com",  
    displayName: "Test User",  
    role: "member",  
    isActive: true,  
    createdAt: new Date(),  
    ...overrides,  
  };  
}
```

What should not live there

- Production code.
- Product logic.
- Runtime validation used by the API.
- Entity definitions.
- Shared UI components used in the product.
- Anything imported by production packages.

Dependency rules

Harness may depend on:

- platform
- packages
- app contracts
- generated types

Production code must not depend on:

- harness

This should be enforced by lint rules.

Testing approach

Harness code should itself be tested. A broken test harness creates false confidence.

Test:

- factories produce valid data;
- fake services match real service contracts;
- API clients match API schemas;
- generated fixture data passes validation.

Build approach

Usually no production build. Type-check it and run tests.

Some harness packages may be buildable if reused across multiple test runners or published internally.

tools/

Purpose

Developer and repository automation.

Examples:

```
tools/  
  generators/  
  codegen/  
  migrations/  
  scripts/  
  lint-rules/  
  release/  
  dev-cli/
```

Tools help engineers change the system safely.

TypeScript code that should live there

- Entity code generators.
- Migration generators.
- Custom lint rules.
- Repo validation scripts.
- Release tooling.
- Dependency graph checks.
- Architecture conformance checks.
- Local developer CLI commands.

Example:

```
await generateEntityArtifacts({
  entityDir: "packages/*/src/entities",
  outputDir: "packages/generated",
});
```

What should not live there

- Runtime code needed by deployed apps.
- Business logic.
- Shared domain types.
- API handlers.
- UI components.

Generated outputs may be production code. The generator itself should not be.

Dependency rules

Tools may depend on:

- platform/entity-core
- platform/entity-codegen
- package metadata
- filesystem/build libraries

Production code must not depend on:

- tools

A generated file can be imported by production code, but a generator runtime should not be.

Testing approach

- Unit tests for generator functions.
- Golden-file tests comparing generated output.
- Integration tests that generate into a temp workspace and run `tsc`.
- Regression tests for migrations.

Build approach

Tools can often run directly with a TypeScript runner during development, but CI should still type-check them. For stable CLI tools, emit JavaScript.

`tsx` is a practical tool for running TypeScript in Node during development, but I would still use `tsc` for type-checking and CI. (github.com)

infra/

Purpose

Infrastructure as code and deployment definitions.

Examples:

```
infra/  
  terraform/  
  cdk/  
  pulumi/  
  docker/  
  k8s/  
  github-actions/  
  environments/
```

This may or may not be TypeScript. If you use AWS CDK or Pulumi, TypeScript may live here. If you use Terraform, much of this is not TypeScript.

TypeScript code that should live there

If using TS-based IaC:

- stack definitions;
- environment models;
- deployment config validation;
- generated environment contracts;
- infrastructure tests;
- policy checks.

Example:

```
type EnvironmentName = "dev" | "staging" | "prod";  
  
type AppDeploymentConfig = {  
  environment: EnvironmentName;  
  apiImageTag: string;  
  webImageTag: string;  
};
```

What should not live there

- Product business logic.
- App runtime code.
- Entity-builder runtime.
- API implementation.
- UI code.

Infra should consume app artifacts, not import app internals.

Dependency rules

Infra may depend on:

- config packages;
- generated deployment metadata;
- platform config validation packages.

Infra should not depend on:

- app implementation internals;
- feature package internals;
- test harness except infra tests.

Testing approach

- Static validation.
- Policy-as-code tests.
- Snapshot tests for synthesized infrastructure.
- Environment config validation.
- Deployment smoke tests.

Build approach

Depends on IaC tool. For TS-based IaC, type-check and compile/bundle according to the tool's expectations.

TypeScript configuration setup

A Kanbien monorepo should not have one magic `tsconfig` that tries to serve every runtime. Backend Node packages, frontend apps, test harnesses, codegen tools, and IaC may need different emit and module behavior.

Use a layered setup.

Recommended config files

```
kanbien/  
  tsconfig.base.json  
  tsconfig.json  
  package.json  
  pnpm-workspace.yaml  
  nx.json  
  
  apps/web/tsconfig.json  
  apps/api/tsconfig.json  
  
  packages/users/tsconfig.json  
  packages/users/tsconfig.test.json  
  
  platform/entity-core/tsconfig.json  
  platform/entity-runtime/tsconfig.json  
  platform/entity-codegen/tsconfig.json
```

Root `tsconfig.base.json`

This holds shared defaults.

```
{  
  "compilerOptions": {  
    "target": "ES2022",  
    "lib": ["ES2022"],  
    "strict": true,  
  
    "noUncheckedIndexedAccess": true,  
    "exactOptionalPropertyTypes": true,  
    "useUnknownInCatchVariables": true,  
    "noImplicitOverride": true,  
    "noFallthroughCasesInSwitch": true,  
    "forceConsistentCasingInFileNames": true,  
  
    "verbatimModuleSyntax": true,  
    "isolatedModules": true,  
  
    "sourceMap": true,  
    "declarationMap": true,  
  
    "skipLibCheck": true  
  }  
}
```

The key setting is:

```
"strict": true
```

For a greenfield platform, this should be non-negotiable. TypeScript's docs recommend enabling strictness for new codebases where possible, because it produces more thorough checks and more accurate tooling. (typescriptlang.org)

I would also enable:


```
"noUncheckedIndexedAccess": true
```

This makes array and object indexing safer.

```
const users: User[] = [];  
const first = users[0];  
// first is User | undefined, not User
```

And:

```
"exactOptionalPropertyTypes": true
```

This makes optional properties stricter. TypeScript's docs describe it as applying stricter rules to properties marked with `?`. (typescriptlang.org)

This matters a lot for API input semantics. In API design, these are not the same thing:

```
{ displayName?: string }
```

versus:

```
{ displayName: string | undefined }
```

The first means "field may be absent." The second means "field is present but may be undefined."

Root `tsconfig.json`

Use this as a solution-style entry point.

```
{  
  "files": [],  
  "references": [  
    { "path": "../platform/entity-core" },  
    { "path": "../platform/entity-runtime" },  
    { "path": "../platform/entity-codegen" },  
    { "path": "../packages/users" },  
    { "path": "../packages/users-api" },  
    { "path": "../apps/api" },  
    { "path": "../apps/web" }  
  ]  
}
```

This file does not compile source itself. It tells TypeScript where the projects are.

Project references let TypeScript split a large program into smaller pieces, improve build times, enforce logical separation, and use `tsc --build`. (typescriptlang.org)

Package-level `tsconfig.json`

Example:

```
{  
  "extends": "../../tsconfig.base.json",  
  "compilerOptions": {  
    "module": "NodeNext",  
    "moduleResolution": "NodeNext",  
  
    "rootDir": "src",  
    "outDir": "dist",  
  }  
}
```

```

    "composite": true,
    "declaration": true,
    "declarationMap": true,

    "tsBuildInfoFile": "dist/.tsbuildinfo"
  },
  "include": ["src/**/*.ts"],
  "references": [
    { "path": "../../platform/entity-runtime" }
  ]
}

```

Use package-level config for anything buildable.

Each buildable package should also have package exports:

```

{
  "name": "@kanbien/users",
  "version": "0.0.0",
  "private": true,
  "type": "module",
  "exports": {
    ".": {
      "types": "./dist/index.d.ts",
      "import": "./dist/index.js"
    }
  },
  "files": ["dist"],
  "scripts": {
    "typecheck": "tsc -p tsconfig.json --noEmit",
    "build": "tsc -b"
  }
}

```

App-level `tsconfig.json`

A frontend app that uses a bundler should not necessarily use the same config as Node packages.

Example `apps/web/tsconfig.json`:

```

{
  "extends": "../../tsconfig.base.json",
  "compilerOptions": {
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "jsx": "react-jsx",
    "noEmit": true
  },
  "include": ["src", "vite.config.ts"]
}

```

TypeScript's docs distinguish `node16 / nodenext` resolution from `bundler` resolution. `bundler` supports package imports / exports but does not require file extensions on relative imports the way Node ESM does. ([typescriptlang.org](https://www.typescriptlang.org))

Example `apps/api/tsconfig.json` for a Node app:

```

{
  "extends": "../../tsconfig.base.json",
  "compilerOptions": {
    "module": "NodeNext",
    "moduleResolution": "NodeNext",

    "rootDir": "src",
    "outDir": "dist",

    "composite": true,
    "declaration": false,

```

```

    "tsBuildInfoFile": "dist/.tsbuildinfo"
  },
  "include": ["src/**/*.ts"],
  "references": [
    { "path": "../../packages/users-api" },
    { "path": "../../platform/api" }
  ]
}

```

Path aliases

Use path aliases carefully.

Bad monorepo pattern:

```

{
  "paths": {
    "@kanbien/users": ["packages/users/src/index.ts"],
    "@kanbien/platform/*": ["platform/*/src/index.ts"]
  }
}

```

This can make the compiler happy while bypassing package boundaries and package exports.

Better:

```

// packages/users/package.json
{
  "name": "@kanbien/users",
  "exports": {
    ".": {
      "types": "../dist/index.d.ts",
      "import": "../dist/index.js"
    }
  }
}

```

Then import like this:

```

import { createUser } from "@kanbien/users";

```

Use package-manager workspaces for linking. pnpm supports monorepo workspaces through a root `pnpm-workspace.yaml`, and its `workspace:` protocol ensures local workspace packages are resolved locally rather than accidentally from the registry. (pnpm.io)

Nx also explicitly recommends moving from TypeScript path aliases to package-manager workspaces and project references for performance benefits. (nx.dev)

Use path aliases mainly for local app ergonomics, not cross-package architecture:

```

{
  "paths": {
    "@app/*": ["./src/*"]
  }
}

```

Project references

Use project references for buildable packages and platform modules.

They are valuable when:

- packages have explicit boundaries;
- builds should be incremental;
- generated artifacts need deterministic compilation;
- CI should only rebuild affected areas;
- platform packages need stable declaration outputs.

But do not blindly add references everywhere if your chosen monorepo tool discourages it. Turborepo's current docs, for example, recommend avoiding TypeScript project references when using Turborepo because they add another configuration and caching layer. ([turborepo.dev](https://turbo.build/repo/docs))

For Kanbien, I would choose **pnpm + Nx + TypeScript project references** because Kanbien sounds like a platform with real internal architecture and generated contracts, not just a handful of apps.

Strict mode

Use strict mode from day one.

Minimum:

```
{
  "compilerOptions": {
    "strict": true
  }
}
```

Recommended Kanbien additions:

```
{
  "compilerOptions": {
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "useUnknownInCatchVariables": true,
    "noImplicitOverride": true,
    "noFallthroughCasesInSwitch": true
  }
}
```

Also establish a governance rule:

```
No new `any` without a comment explaining why.
No unsafe type assertion around external input.
No `// @ts-ignore` without an issue link.
```

Module resolution

Use two defaults:

For Node-emitted packages:

```
{
  "compilerOptions": {
    "module": "NodeNext",
    "moduleResolution": "NodeNext"
  }
}
```

For browser-bundled apps:

```
{
  "compilerOptions": {
    "module": "ESNext",
    "moduleResolution": "Bundler"
  }
}
```

With Node ESM, TypeScript follows Node's stricter module behavior. In ESM packages, relative imports need full output extensions, so TypeScript source often imports `./foo.js` even though the source file is `foo.ts`. TypeScript's 4.7 release notes explain this Node ESM behavior directly. (typescriptlang.org)

Example:

```
// In src/index.ts
export { defineEntity } from "./define-entity.js";
```

This looks odd at first, but it produces correct Node ESM output.

ESM vs CommonJS

For Kanbien greenfield code, choose **ESM-first**.

Use:

```
{
  "type": "module"
}
```

CommonJS should be the exception, mainly for:

- legacy tooling;
- specific test or build tools that require CJS;
- compatibility packages;
- `.cjs` config files when required.

Do not dual-publish ESM and CommonJS internally unless you have a real consumer need. Dual publishing adds complexity.

Recommendation:

```
Internal Kanbien packages: ESM only.
External SDK package, if needed: consider dual output only when customers need CJS.
Tooling config: use .cjs only where the tool requires it.
```

Linting

Use ESLint with `typescript-eslint`.

ESLint identifies code patterns and helps avoid bugs and consistency problems. (eslint.org)

Use type-aware linting selectively:

- enabled for `platform/` and important `packages/`;
- enabled in CI;
- possibly lighter in local fast loops.

With `typescript-eslint v8+`, use `parserOptions.projectService` for typed linting so linting uses the same TypeScript project information as editors. (typescript-eslint.io)

Also use Nx module-boundary rules:

```
'@nx/enforce-module-boundaries': [
  'error',
  {
    depConstraints: [
      {
        sourceTag: 'scope:app',
        onlyDependOnLibsWithTags: ['scope:package', 'scope:platform']
      },
      {
        sourceTag: 'scope:package',
        onlyDependOnLibsWithTags: ['scope:package', 'scope:platform']
      },
      {
        sourceTag: 'scope:platform',
        onlyDependOnLibsWithTags: ['scope:platform']
      },
      {
        sourceTag: 'scope:harness',
        onlyDependOnLibsWithTags: ['scope:package', 'scope:platform', 'scope:app']
      }
    ]
  }
]
```

Nx's module-boundary docs describe this exact category of control: preventing unplanned cross-dependencies by checking imports and package dependencies during linting. (nx.dev)

Formatting

Use Prettier.

Do not argue about formatting in code review. Let the tool do it.

Prettier describes itself as an opinionated formatter that removes original styling and reprints code consistently. (prettier.io)

Recommended:

```
{
  "scripts": {
    "format": "prettier . --write",
    "format:check": "prettier . --check"
  }
}
```

Type checking

Do not rely on app builds to type-check everything.

Use explicit type-check commands:

```
{
  "scripts": {
    "typecheck": "nx run-many -t typecheck"
  }
}
```

Per package:

```
{
  "scripts": {
    "typecheck": "tsc -p tsconfig.json --noEmit"
  }
}
```

For referenced build graphs:

```
tsc -b
```

In CI:

```
pnpm nx affected -t typecheck lint test build
```

Nx's affected command determines the minimum affected project set and runs tasks only for those projects, which is important as a monorepo grows. (nx.dev)

Build outputs

Use standard build outputs:

```
dist/
  index.js
  index.d.ts
  index.js.map
  index.d.ts.map
```

For packages:

- emit JavaScript if the package is consumed by Node;
- emit `.d.ts` declarations;
- expose only public entry points through `package.json` `exports`.

For frontend apps:

- let the app framework produce its output;
- use `tsc --noEmit` or framework type-checking for validation.

For generated code:

```
packages/users/src/generated/
  user.schema.ts
  user.types.ts
  user.api.ts
  user.form.ts
  user.table.ts
```

or:

```
packages/generated/users/
  src/
  dist/
```

My preference: generated code belongs in generated packages when it is shared broadly, and package-local generated folders when it is owned by one feature.

Recommended clean setup for Kanbien

Package manager

Use **pnpm**.

Root:

```
# pnpm-workspace.yaml
packages:
  - "apps/*"
  - "packages/*"
  - "platform/*"
  - "harness/*"
  - "tools/*"
  - "infra/*"
```

Use workspace dependencies:

```
{
  "dependencies": {
    "@kanbien/entity-runtime": "workspace:*"
  }
}
```

Reason: deterministic monorepo linking, efficient installs, and clear local package relationships.

Monorepo tooling

Use **Nx**.

Reason:

- dependency graph;
- affected builds;
- local and remote caching;
- boundary enforcement;
- project tagging;
- CI optimization;
- good fit for enterprise monorepos.

I would use Nx as the task orchestrator, not necessarily as a heavy code-generation framework for everything.

TypeScript compiler settings

Use:

```
TypeScript 6.x stable
strict: true
ESM-first
NodeNext for Node packages
Bundler for frontend apps
project references for buildable packages
declaration output for packages
noEmit for bundled apps
```


Do not adopt TypeScript 7 native previews as the default compiler for Kanbien production until stable. Track it in a compatibility branch.

Linting

Use:

```
ESLint
typescript-eslint
@nx/enforce-module-boundaries
custom Kanbien architectural rules if needed
```

Add custom rules later for things like:

- no production import from harness ;
- no app import from another app;
- no raw `process.env` outside config packages;
- no direct database import outside repositories;
- no unsafe `as User` from API input;
- no editing generated files.

Formatting

Use:

```
Prettier
EditorConfig
format check in CI
```

Testing

Use:

```
Vitest for unit and integration tests
Playwright or equivalent for browser E2E
contract tests around API schemas
type-level tests for entity inference
golden-file tests for code generation
```

Vitest is a modern Vite-powered test framework with out-of-the-box ESM, TypeScript, and JSX support, and it works for backend code as well. (vitest.dev)

Code generation

Use a Kanbien-owned generator.

For the entity-builder, avoid clever runtime reflection. Prefer:

```
entity definition as typed metadata
-> generator reads metadata
-> emits schemas, types, DB metadata, API contracts, UI config, tests
-> CI checks generated output
```

The generator should be deterministic:

```
pnpm generate
git diff --exit-code
pnpm typecheck
pnpm test
```

CI checks

Minimum CI pipeline:

```
pnpm install --frozen-lockfile
pnpm format:check
pnpm generate
git diff --exit-code
pnpm nx affected -t lint typecheck test build --base=origin/main --head=HEAD
```

For main branch:

```
pnpm nx run-many -t lint typecheck test build
```

Also add:

```
API contract diff check
DB migration check
generated artifact freshness check
dependency boundary check
security/dependency scan
```

How TypeScript should support the entity-builder idea

Your desired pipeline is:

```
Entity definition
→ validation rules
→ generated types
→ backend schema
→ API contract
→ frontend form/table config
→ tests
```

The most important design decision is the source of truth.

I would make the **entity definition** the source of truth, but I would not make it a loose JSON blob. It should be a TypeScript-checked declarative object.

Example concept:

```
export const UserEntity = defineEntity({
  name: "User",
  table: "users",
  fields: {
    id: {
      kind: "uuid",
      required: true,
      readonly: true
    },
    email: {
      kind: "email",
      required: true,
      unique: true,
      pii: true
    },
    displayName: {
      kind: "string",
      required: true,
      minLength: 1,
      maxLength: 120
    }
  }
});
```

The definition drives all downstream artifacts.

What TypeScript enforces

TypeScript can enforce that:

- field definitions use legal field kinds;
- required properties are present;
- enum defaults are valid enum values;
- form config references real entity fields;
- table config references real output fields;
- API handlers return the declared output shape;
- repository functions accept the declared persistence input;
- generated types match generated schemas;
- invalid states are unrepresentable where modeled.

Example:

```
const columns = [
  { key: "email", label: "Email" },
  { key: "displayName", label: "Name" },
  { key: "doesNotExist", label: "Broken" } // TypeScript error
] satisfies ColumnConfig<UserOutput>[];
```

If `doesNotExist` is not a key of `UserOutput`, this fails at compile time.

What TypeScript cannot enforce alone

TypeScript cannot prove that:

- an HTTP request body is valid;
- a database row actually has the expected shape;
- a migration was applied;
- a generated SQL column exists in production;
- an authorization policy is correct;
- a user-entered email is real;
- a frontend form was filled out honestly;
- an API client from another language followed the contract.

So the entity-builder should combine:

```
TypeScript compile-time checks
+ runtime validation
+ generated tests
+ migration checks
+ contract tests
+ CI freshness checks
```

Worked User example

Below is a simplified but enterprise-grade pattern.

1. Entity definition

File:

```
packages/users/src/entities/user.entity.ts
```

```
import { defineEntity } from "@kanbien/entity-core";

export const UserEntity = defineEntity({
  name: "User",
  table: "users",

  fields: {
    id: {
      kind: "uuid",
      required: true,
      readonly: true
    },
    email: {
      kind: "email",
      required: true,
      unique: true,
      pii: true,
      label: "Email"
    },
    displayName: {
      kind: "string",
      required: true,
      minLength: 1,
      maxLength: 120,
      label: "Display name"
    },
    role: {
      kind: "enum",
      required: true,
      values: ["admin", "member"] as const,
      default: "member",
      label: "Role"
    },
    isActive: {
      kind: "boolean",
      required: true,
      default: true,
      label: "Active"
    },
    createdAt: {
      kind: "datetime",
      required: true,
      readonly: true
    }
  }
});
```

The platform type behind `defineEntity` might look roughly like this:

```
export type FieldDefinition =
  | {
    kind: "uuid";
    required?: boolean;
    readonly?: boolean;
```

```

        unique?: boolean;
        pii?: boolean;
        label?: string;
    }
    | {
        kind: "email";
        required?: boolean;
        readonly?: boolean;
        unique?: boolean;
        pii?: boolean;
        label?: string;
    }
    | {
        kind: "string";
        required?: boolean;
        readonly?: boolean;
        minLength?: number;
        maxLength?: number;
        label?: string;
    }
    | {
        kind: "boolean";
        required?: boolean;
        readonly?: boolean;
        default?: boolean;
        label?: string;
    }
    | {
        kind: "datetime";
        required?: boolean;
        readonly?: boolean;
        label?: string;
    }
    | {
        kind: "enum";
        required?: boolean;
        readonly?: boolean;
        values: readonly [string, ...string[]];
        default?: string;
        label?: string;
    };
};

export type EntityDefinition = {
    name: string;
    table: string;
    fields: Record<string, FieldDefinition>;
};

export function defineEntity<const TEntity extends EntityDefinition>(
    entity: TEntity
): TEntity {
    return entity;
}

```

The `const` generic preserves literal field names and enum values. This is what lets TypeScript later know that `"email"`, `"displayName"`, and `"role"` are actual fields.

2. Inferred TypeScript type

The platform can infer a type from the entity.

```

type ScalarValue<TField> =
    TField extends { kind: "uuid" } ? string :
    TField extends { kind: "email" } ? string :
    TField extends { kind: "string" } ? string :
    TField extends { kind: "boolean" } ? boolean :
    TField extends { kind: "datetime" } ? Date :
    TField extends { kind: "enum"; values: readonly (infer V)[] } ? V :
    never;

type RequiredFieldNames<TFields> = {
    [K in keyof TFields]:
        TFields[K] extends { required: true } ? K : never
}[keyof TFields];

```

```

type OptionalFieldNames<TFields> =
  Exclude<keyof TFields, RequiredFieldNames<TFields>>;

export type EntityModel<TEntity extends EntityDefinition> =
  {
    [K in RequiredFieldNames<TEntity["fields"]>]:
      ScalarValue<TEntity["fields"][K]>
  } & {
    [K in OptionalFieldNames<TEntity["fields"]>]?:
      ScalarValue<TEntity["fields"][K]>
  };

```

Then:

```

import type { EntityModel } from "@kanbien/entity-core";
import { UserEntity } from "../user.entity.js";

export type User = EntityModel<typeof UserEntity>;

```

The inferred `User` is effectively:

```

type User = {
  id: string;
  email: string;
  displayName: string;
  role: "admin" | "member";
  isActive: boolean;
  createdAt: Date;
};

```

No engineer hand-wrote that type. It came from the entity definition.

3. Generated validation schema

Generated file:

```

packages/users/src/generated/user.validation.ts

```

```

// GENERATED FILE. DO NOT EDIT.

import { z } from "zod";

export const UserRoleSchema = z.enum(["admin", "member"]);

export const UserSchema = z.object({
  id: z.string().uuid(),
  email: z.string().email(),
  displayName: z.string().min(1).max(120),
  role: UserRoleSchema,
  isActive: z.boolean(),
  createdAt: z.date()
});

export const UserCreateInputSchema = z.object({
  email: z.string().email(),
  displayName: z.string().min(1).max(120),
  role: UserRoleSchema.default("member")
});

export const UserUpdateInputSchema = z.object({
  email: z.string().email().optional(),
  displayName: z.string().min(1).max(120).optional(),
  role: UserRoleSchema.optional(),
  isActive: z.boolean().optional()
});

```

And generated types:

```
// GENERATED FILE. DO NOT EDIT.

import type { z } from "zod";
import {
  UserSchema,
  UserCreateInputSchema,
  UserUpdateInputSchema
} from "../user.validation.js";

export type User = z.infer<typeof UserSchema>;
export type UserCreateInput = z.infer<typeof UserCreateInputSchema>;
export type UserUpdateInput = z.infer<typeof UserUpdateInputSchema>;
```

The key advantage: runtime schema and TypeScript type do not drift.

4. API input/output contract

Generated or package-owned file:

```
packages/users-api/src/user.contract.ts
```

```
import { z } from "zod";
import {
  UserCreateInputSchema,
  UserUpdateInputSchema
} from "@kanbien/users/generated/user.validation";

export const UserOutputSchema = z.object({
  id: z.string().uuid(),
  email: z.string().email(),
  displayName: z.string(),
  role: z.enum(["admin", "member"]),
  isActive: z.boolean(),
  createdAt: z.string().datetime()
});

export const CreateUserContract = {
  method: "POST",
  path: "/users",
  input: UserCreateInputSchema,
  output: UserOutputSchema
} as const;

export const UpdateUserContract = {
  method: "PATCH",
  path: "/users/:id",
  input: UserUpdateInputSchema,
  output: UserOutputSchema
} as const;
```

Then an API handler:

```
import {
  CreateUserContract,
  UserOutputSchema
} from "../user.contract.js";

export const createUserRoute = defineRoute(CreateUserContract, async ({
  input,
  ctx
}) => {
  const user = await ctx.users.create(input);

  return UserOutputSchema.parse({
    id: user.id,
    email: user.email,
    displayName: user.displayName,
    role: user.role,
    isActive: user.isActive,
    createdAt: user.createdAt.toISOString()
  });
});
```



```
});  
});
```

Here TypeScript helps at compile time, and Zod validates at runtime.

If the handler forgets `displayName`, the output parse fails in tests and runtime. If the handler returns the wrong type, TypeScript and validation both help.

5. Persistence model

Generated file:

```
packages/users/src/generated/user.persistence.ts
```

```
// GENERATED FILE. DO NOT EDIT.  
  
export type UserRow = {  
  id: string;  
  email: string;  
  display_name: string;  
  role: "admin" | "member";  
  is_active: boolean;  
  created_at: Date;  
};  
  
export type UserInsertRow = {  
  id: string;  
  email: string;  
  display_name: string;  
  role: "admin" | "member";  
  is_active: boolean;  
  created_at: Date;  
};  
  
export type UserUpdateRow = Partial<{  
  email: string;  
  display_name: string;  
  role: "admin" | "member";  
  is_active: boolean;  
}>;
```

Repository code:

```
import type {  
  User,  
  UserCreateInput  
} from "../generated/user.types.js";  
  
import type {  
  UserInsertRow,  
  UserRow  
} from "../generated/user.persistence.js";  
  
function toUser(row: UserRow): User {  
  return {  
    id: row.id,  
    email: row.email,  
    displayName: row.display_name,  
    role: row.role,  
    isActive: row.is_active,  
    createdAt: row.created_at  
  };  
}  
  
function toInsertRow(input: UserCreateInput): UserInsertRow {  
  return {  
    id: crypto.randomUUID(),  
    email: input.email,  
    display_name: input.displayName,  
    role: input.role,  
  };  
}
```

```

    is_active: true,
    created_at: new Date()
  };
}

```

If someone renames `displayName` to `fullName` in the entity definition and regenerates, these mappers fail to compile until updated.

That is exactly what you want.

6. Frontend form config

Generated or semi-generated file:

```
packages/users/src/generated/user.form.ts
```

```

import type { FormConfig } from "@kanbien/ui-forms";
import type { UserCreateInput } from "../user.types.js";

export const UserCreateFormConfig = {
  fields: {
    email: {
      label: "Email",
      input: "email",
      required: true
    },

    displayName: {
      label: "Display name",
      input: "text",
      required: true,
      minLength: 1,
      maxLength: 120
    },

    role: {
      label: "Role",
      input: "select",
      required: true,
      options: [
        { label: "Admin", value: "admin" },
        { label: "Member", value: "member" }
      ]
    }
  ]
} satisfies FormConfig<UserCreateInput>;

```

If a frontend engineer adds:

```

department: {
  label: "Department",
  input: "text"
}

```

but `department` is not part of `UserCreateInput`, TypeScript should reject it if `FormConfig<T>` is designed correctly.

7. Frontend table config

```

import type { TableConfig } from "@kanbien/ui-table";
import type { User } from "../user.types.js";

export const UserTableConfig = {
  columns: [
    { key: "email", label: "Email" },
    { key: "displayName", label: "Name" },

```

```

    { key: "role", label: "Role" },
    { key: "isActive", label: "Active" },
    { key: "createdAt", label: "Created" }
  ]
} satisfies TableConfig<User>;

```

If someone writes:

```
{ key: "name", label: "Name" }
```

but the field is really `displayName`, TypeScript catches it.

8. Generated tests

Generated file:

```
packages/users/src/generated/user.generated.test.ts
```

```

// GENERATED FILE. DO NOT EDIT.

import { describe, expect, test } from "vitest";
import {
  UserCreateInputSchema,
  UserOutputSchema
} from "../user.validation.js";

describe("User generated schemas", () => {
  test("accepts a valid create input", () => {
    const parsed = UserCreateInputSchema.parse({
      email: "alice@example.com",
      displayName: "Alice",
      role: "member"
    });

    expect(parsed.email).toBe("alice@example.com");
  });

  test("rejects an invalid email", () => {
    expect(() =>
      UserCreateInputSchema.parse({
        email: "not-an-email",
        displayName: "Alice",
        role: "member"
      })
    ).toThrow();
  });

  test("accepts a valid API output", () => {
    const parsed = UserOutputSchema.parse({
      id: "3f4d5f0e-1234-4b33-9234-38db7f7b3c2a",
      email: "alice@example.com",
      displayName: "Alice",
      role: "member",
      isActive: true,
      createdAt: new Date().toISOString()
    });

    expect(parsed.role).toBe("member");
  });
});

```

Type-level test:

```

import { expectTypeOf, test } from "vitest";
import type { UserCreateInput } from "../user.types.js";

test("UserCreateInput has expected field types", () => {
  expectTypeOf<UserCreateInput>().toMatchTypeOf<{
    email: string;

```

```
    displayName: string;  
    role?: "admin" | "member";  
  }>();  
});
```

This proves two things:

1. runtime validation behaves as expected;
2. generated TypeScript types have the expected shape.

The key design lesson

For Kanbien, the entity-builder should not simply "generate TypeScript types."

It should generate and verify a **contract chain**:

```
UserEntity
  produces User type
  produces UserCreateInput type
  produces UserCreateInputSchema
  produces UserRow / UserInsertRow
  produces API input/output schemas
  produces frontend form config
  produces frontend table config
  produces generated tests
  participates in CI drift checks
```

The payoff is huge: when the entity changes, the platform tells you exactly what broke.

That is the right use of TypeScript in enterprise software. Not decoration. Not autocomplete theater. A real, enforced architecture.